LeetCode HOT 100

PART 12

Stack

栈

69. 有效的括号 (No.20) EASY

给定一个只包括 `'('`，`')'`，`'{'`，`'}'`，`'['`，`']'` 的字符串 `s` ，判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

示例 1：

> **输入：** s = "()"
>
> **输出：** true

示例 2：

> **输入：** s = "()[]{}"
>
> **输出：** true

示例 3：

> **输入：** s = "(]"
>
> **输出：** false

示例 4：

> **输入：** s = "([])"
>
> **输出：** true

示例 5：

> **输入：** s = "([)]"
>
> **输出：** false

提示：

- `1 <= s.length <= 10^4`
- `s` 仅由括号 `'()[]{}'` 组成

69. 有效的括号 (No.20) EASY

思路

左括号入栈，右括号弹栈。如果弹栈时栈空或者栈顶左括号和当前右括号不匹配则不合法，且遍历结束后栈非空也不合法，除此之外合法。

```python
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        char_map = {
            '(': ')',
            '[': ']',
            '{': '}'
        }
        for char in s:
            if char in set(['(', '[', '{']):
                stack.append(char)  # 遇到左括号入栈
            else:
                if not stack or char_map[stack.pop()] != char:
                    return False
        return False if stack else True
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(n)$

70. 最小栈 (No.155) MEDIUM

设计一个支持 `push` ， `pop` ， `top` 操作，并能在常数时间内检索到最小元素的栈。

实现 `MinStack` 类:

- `MinStack()` 初始化堆栈对象。
- `void push(int val)` 将元素val推入堆栈。
- `void pop()` 删除堆栈顶部的元素。
- `int top()` 获取堆栈顶部的元素。
- `int getMin()` 获取堆栈中的最小元素。

示例 1:

输入:
```
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]
```

输出:
```
[null,null,null,null,-3,null,0,-2]
```

解释:
```
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin();   --> 返回 -3.
minStack.pop();
minStack.top();      --> 返回 0.
minStack.getMin();   --> 返回 -2.
```

提示:

- $-2^{31} <= val <= 2^{31} - 1$
- `pop`、`top` 和 `getMin` 操作总是在 **非空栈** 上调用
- `push`, `pop`, `top`, and `getMin` 最多被调用 $3 * 10^4$ 次

70. 最小栈 (No.155) MEDIUM

思路

每一个栈的状态对应的最小值，其实是栈中元素序列的前缀最小值：

`f[i] = min(f[i-1],val)`

换言之，每个栈中元素作为栈顶时，其最小元素必不在其上方。

因此只需要一个辅助栈`min_stack`来记录每个元素作为栈顶时对应的最小值是多少。

辅助栈初始放一个`inf`哨兵防止栈空报错。

时间复杂度：所有操作都是$\mathcal{O}(1)$

空间复杂度：$\mathcal{O}(n)$

```python
class MinStack:

    def __init__(self):
        self.stack = []
        self.min_stack = [inf]  # 辅助栈，栈底哨兵

    def push(self, val: int) -> None:
        self.stack.append(val)
        self.min_stack.append(min(self.min_stack[-1], val))

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]


# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(val)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()
```

71. 字符串解码 (No.394) MEDIUM

给定一个经过编码的字符串，返回它解码后的字符串。

编码规则为：`k[encoded_string]`，表示其中方括号内部的 `encoded_string` 正好重复 `k` 次。注意 `k` 保证为正整数。

你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。

此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 `k` ，例如不会出现像 `3a` 或 `2[4]` 的输入。

测试用例保证输出的长度不会超过 10^5。

示例 1：

> **输入：** `s = "3[a]2[bc]"`
> **输出：** `"aaabcbc"`

示例 2：

> **输入：** `s = "3[a2[c]]"`
> **输出：** `"accaccacc"`

示例 3：

> **输入：** `s = "2[abc]3[cd]ef"`
> **输出：** `"abcabccdcdcdef"`

示例 4：

> **输入：** `s = "abc3[cd]xyz"`
> **输出：** `"abccdcdcdxyz"`

提示：

- `1 <= s.length <= 30`
- `s` 由小写英文字母、数字和方括号 `'[]'` 组成
- `s` 保证是一个 **有效** 的输入。
- `s` 中所有整数的取值范围为 `[1, 300]`

71. 字符串解码 (No.394) MEDIUM

Solution 1 – 模拟递归

思路

用栈模拟递归，当遇到左括号的时候，应先放下手头所有的字符串res和数字num，扔到一边先去完成括号里面的子任务。因此递归时需要用栈保存现场，等子任务完成，得到一个子任务的结果res时，再取栈顶pre_res,pre_num还原现场，并把res重复pre_num次的子串拼在pre_res后面。

时间复杂度：$\mathcal{O}(n+m)$，n为s长度，m为解码后字符串长度；

空间复杂度：$\mathcal{O}(m)$

```python
class Solution:
    def decodeString(self, s: str) -> str:
        stack = []
        res = ''
        num = 0  # 用于存储数字
        for char in s:
            if char.isalpha():
                res += char
            elif char.isdigit():
                num = num * 10 + int(char)
            elif char == '[':  # 左括号递归
                # 保存现场并重置
                stack.append((res, num))
                res, num = '', 0
            else:  # 右括号回溯
                # 还原现场
                pre_res, pre_num = stack.pop()
                res = pre_res + res * pre_num
        return res
```

71. 字符串解码 (No.394) MEDIUM

Solution 2 – 模拟扫描

思路

只需要区分是否是右括号，非右括号全部压栈，遇到右括号先弹出所有字母，逆置后得到字母子串alpha，然后弹出左括号，最后弹出数字，逆置后得到num，alpha*num重新入栈。最终栈中存储的是所有最外层的字母子串，再拼接起来得到答案。

时间复杂度：$\mathcal{O}(n+m)$

空间复杂度：$\mathcal{O}(m)$

```python
class Solution:
    def decodeString(self, s: str) -> str:
        stack = []
        for char in s:
            if char != ']':  # 不是右括号全部入栈
                stack.append(char)
            else:  # 遇到右括号
                # 提取字母子串
                alpha = []
                while stack[-1].isalpha():
                    alpha.append(stack.pop())
                alpha = "".join(reversed(alpha))  # 字母逆置
                stack.pop()  # 弹出左括号
                # 提取数字
                num = []
                while stack and stack[-1].isdigit():
                    num.append(stack.pop())
                num = int("".join(reversed(num)))  # 数字逆置
                stack.append(alpha * num)
        return "".join(stack)
```

72. 每日温度 (No.739) MEDIUM

给定一个整数数组 `temperatures` ，表示每天的温度，返回一个数组 `answer` ，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

示例 1:

> **输入：** `temperatures = [73,74,75,71,69,72,76,73]`
> **输出：** `[1,1,4,2,1,1,0,0]`

示例 2:

> **输入：** `temperatures = [30,40,50,60]`
> **输出：** `[1,1,1,0]`

示例 3:

> **输入：** `temperatures = [30,60,90]`
> **输出：** `[1,1,0]`

提示：

- `1 <= temperatures.length <=` 10^5
- `30 <= temperatures[i] <= 100`

72. 每日温度 (No.739) MEDIUM

思路

从某个位置开始找接下去第一个满足某条件的元素，很典型的单调栈应用场景，本题单调栈中的温度是递减的。对于从左到右遍历的情况，由于遍历时还不知道接下去最高温什么时候出现，因此把下标入栈记住位置。等到有新元素t开始清栈的时候，这些被清理的位置下一个高温就是t对应时刻。

这道题也可以从右往左遍历，单调栈仍然温度递减。变化在于新元素t清栈后，栈顶元素（如果有）是t的下一个高温。

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        stack = []  # 单调栈，栈顶是上一个比当前温度高的气温的下标
        n = len(temperatures)
        res = [0] * n
        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                res[stack[-1]] = i - stack[-1]
                stack.pop()
            stack.append(i)
        return res
```

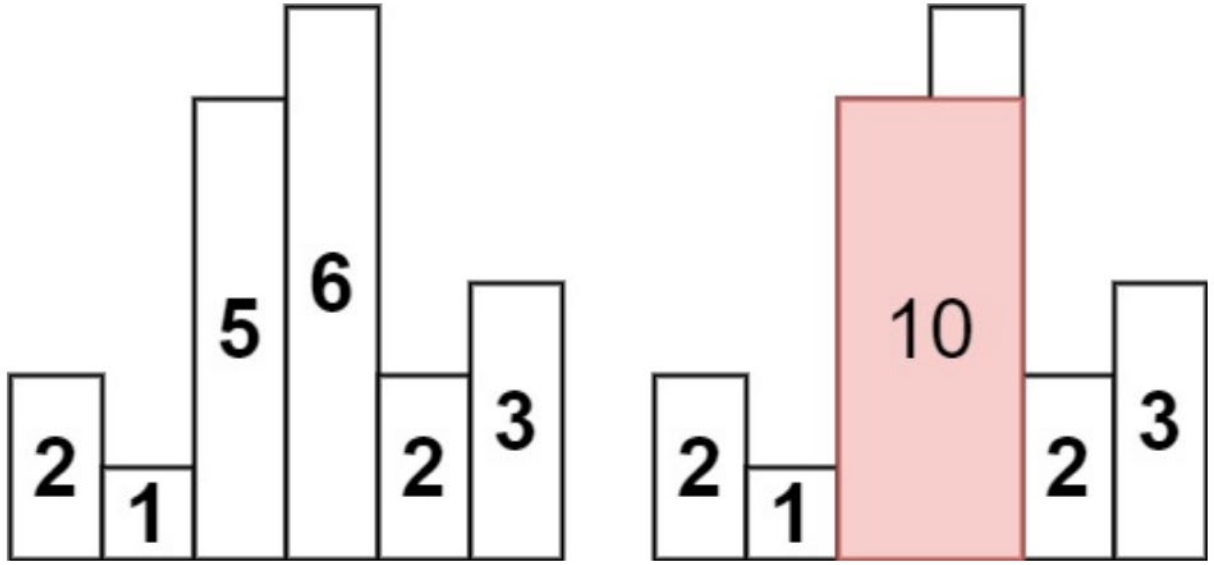
时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(n)$

73. 柱状图中最大的矩形 (No.84) HARD

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1 。

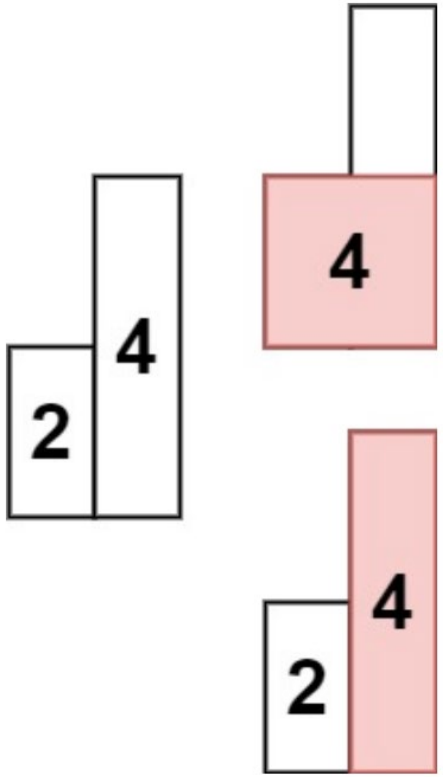
求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1:



输入: heights = [2,1,5,6,2,3]
输出: 10
解释: 最大的矩形为图中红色区域，面积为 10

示例 2:



输入: heights = [2,4]
输出: 4

提示:

- `1 <= heights.length <=`10^5
- `0 <= heights[i] <=` 10^4

73. 柱状图中最大的矩形 (No.84) HARD

Solution 1 – 枚举最矮柱子

思路

根据木桶效应，枚举每个柱子作为最矮的柱子，向两侧展开计算最大矩形。

为了计算两侧展开的臂长，需要知道每个柱子i向左和向右遇到的第一个比自己矮的柱子left[i]和right[i]，这样以heights[i]为高，right[i]-left[i]-1为宽的矩形就是最大的，用单调栈就可以实现计算left和right。

left和right初始化-1和n保证没有某侧没有比自己更矮的柱子时也不会出错。

```python
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        stack = []  # 单调栈，存储下标，高度递增
        # 从左往右和从右往左第一个比自己矮的柱子
        n = len(heights)
        left, right = [-1] * n, [n] * n
        # 从左往右遍历
        for i in range(n):
            while stack and heights[stack[-1]] > heights[i]:
                x = stack.pop()
                # 让x出栈的元素i是x右侧第一个矮柱子
                right[x] = i
            if stack:  # 如果栈非空，栈顶是i左侧第一个矮柱子
                left[i] = stack[-1]
            stack.append(i)
        # 枚举最矮柱子计算最大面积
        res = 0
        for i in range(n):
            res = max(res, heights[i] * (right[i] - left[i] - 1))
        return res
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(n)$

73. 柱状图中最大的矩形 (No.84) HARD

Solution 2 – 枚举右边界

思路

Solution1用了两趟遍历，其实也可以一趟遍历从左往右做完。对于遍历到柱子i，x如果被i弹栈了，i就是x作为最矮柱子的右边界+1的位置，x弹栈后如果栈非空，那么栈顶stack[-1]对应的是x左边界-1的位置，因此一趟遍历就能找到左右边界了。不过注意有的柱子i可能没有左右边界，那么维护一个哨兵heights[-1]=0来确保：①x被i弹栈时如果栈空，说明x没有左边界，栈底放一个-1保证其一定有左边界，此时宽刚好是i；②i入栈后之后没人让它弹栈，说明i没有右边界，它会滞留栈中，最后一个元素由于高度为0必将其弹栈，保证其一定有右边界，此时宽是n-1-stack[-1]。

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(n)$

```python
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        stack = [-1]  # 单调栈，存储下标，高度递增，哨兵-1防止有元素没有左边界
        res = 0
        heights.append(0)  # 哨兵，防止有元素没有左右边界
        for i in range(len(heights)):
            while stack and heights[stack[-1]] > heights[i]:
                x = stack.pop()  # x被i弹出，i是x的右边界+1
                # 此时如果栈非空，stack[-1]对应是x的左边界-1
                area = (i-stack[-1]-1) * heights[x]
                res = max(res, area)
            stack.append(i)
        return res
```